A Rust crate and command-line tool · `crates.io/crates/parachuter` · v0.2.0

parachuter

Reliable, rate-limited file downlink over lossy, one-way UDP links. It gets every byte down, and never sends anything twice unless it was lost.

32 B	1 bit	0
header on every packet	per chunk to track a file	restarts to retune a link

Built for and first flown on **SuperBIT**, a stratospheric balloon telescope. One Rust binary runs every role, from the flight computer to the ground station.

---

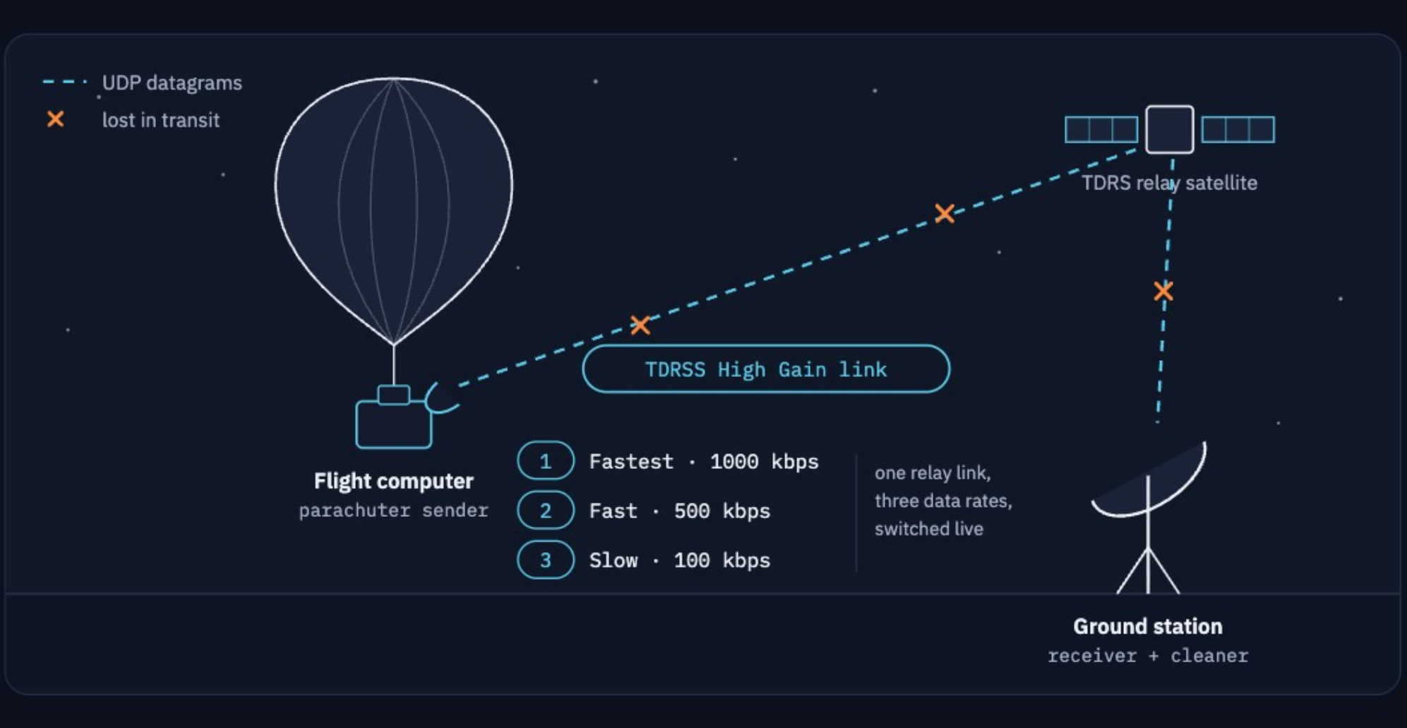
§1 The problem

The sky is a terrible network.

A balloon payload produces **multi-gigabyte science images**. It comes down through a NASA TDRS relay satellite on a high-gain link that is narrow and lossy, and the uplink budget goes to commands and telemetry.

So the ground can't acknowledge every packet the way TCP needs. Delivery has to be **fire and forget**, plus a tiny side channel to ask for whatever went missing.

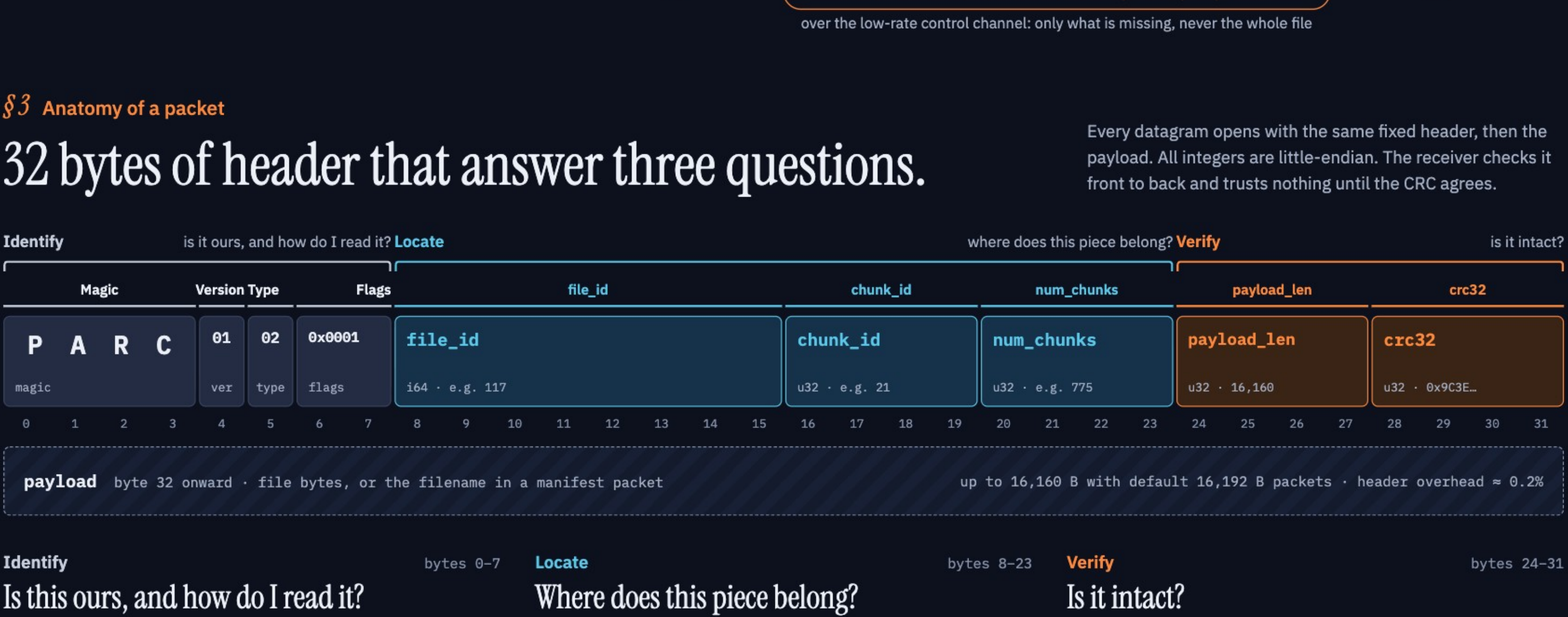
1	**TDRSS High Gain** (Fastest)	`1000 kbps`
2	**TDRSS High Gain** (Fast)	`500 kbps`
3	**TDRSS High Gain** (Slow)	`100 kbps`



§2 How it works

Send everything once. Then send only the gaps.

Five stages, one loop. The loop repeats until the ground holds every chunk, then the file lands byte for byte identical.



01 **Ingest**
Drop a file in a watched priority folder, or hand one over by path on the control socket. A SQLite ledger records every file.
`priority_dirs · ctl submit`

02 **Chunk**
Split the file into numbered datagrams, streamed from disk. Each carries a 32-byte header and a CRC-32 over the whole packet.
`16,192 B default · 64 to 65,000`

03 **Pace & send**
A token bucket holds the stream to the active link's budget. Switch links by name, mid-transfer, without a restart.
`max_kbps · set-link`

04 **Reassemble**
Check magic and CRC, write the bytes straight to their offset, flip one bit in the manifest. Arrival order doesn't matter.
`holding/ → downloads/`

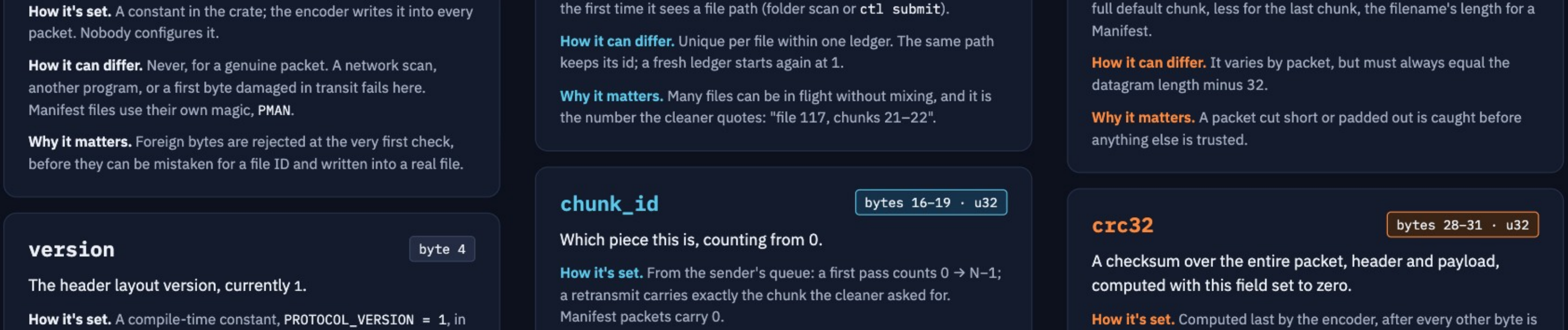
05 **Clean**
Read every bitmap, find the gaps, and ask the sender for exactly those chunk ranges. Nothing already received is re-sent.
`3 dedup guards · see 07`

over the low-rate control channel: only what is missing, never the whole file

§3 Anatomy of a packet

32 bytes of header that answer three questions.

Every datagram opens with the same fixed header, then the payload. All integers are little-endian. The receiver checks it front to back and trusts nothing until the CRC agrees.



payload `byte 32 onward · file bytes, or the filename in a manifest packet` `up to 16,160 B with default 16,192 B packets · header overhead ≈ 0.2%`

Identify `bytes 0–7`
Is this ours, and how do I read it?

magic · "PARC" `bytes 0–3`

Four fixed ASCII letters, `50 41 52 43`, that open every packet.

How it's set. A constant in the crate; the encoder writes it into every packet. Nobody configures it.

How it can differ. Never, for a genuine packet. A network scan, another program, or a first byte damaged in transit fails here. Manifest files use their own magic, `PMAN`.

Why it matters. Foreign bytes are rejected at the very first check, before they can be mistaken for a file ID and written into a real file.

version `byte 4`

The header layout version, currently `1`.

How it's set. A compile-time constant, `PROTOCOL_VERSION = 1`, in whichever parachuter release is running. It is not in the config, and `ctl` can't change it.

How it can differ. Only when a release changes the header layout (a field added, widened or moved): that release stamps **2**. Flight code is frozen at launch while the ground keeps updating, so the two ends can disagree. Today's receiver accepts exactly 1 and drops the rest as "unsupported version", so a mismatch fails loudly instead of misreading every field; a later ground release could read 1 and 2 side by side. Changing chunk size, link or rate never needs a new version.

Why it matters. Old and new software can disagree safely, and the protocol has a clean upgrade path.

type `byte 5`

What the payload means: 1 Data · 2 Manifest (filename) · 3 Retransmit · 4 NameOnly · 5 Heartbeat.

How it's set. Per queue item. A first pass sends a Manifest, then Data for every chunk; anything the cleaner asks for goes out as Retransmit, and a re-requested filename as another Manifest.

How it can differ. NameOnly and Heartbeat are understood by the receiver but not sent by today's sender. Values outside 1–5 are dropped, so new types can be added without a version bump.

Why it matters. The receiver takes the right path (write bytes, record a name, or ignore) with no guessing.

flags `bytes 6–7`

Sixteen on/off bits. Three are defined:

`LAST_CHUNK` `bit 0`	`in use`
`COMPRESSED` `bit 1`	`not set today`
`RETRANSMIT_HINT` `bit 2`	`in use`

How it's set. The chunker sets LAST_CHUNK on a file's final chunk and RETRANSMIT_HINT on every retransmit. COMPRESSED is defined but not set today, even for `.fits.bz2` files.

How it can differ. Typical values: `0x0000` ordinary · `0x0001` last chunk · `0x0004` retransmit · `0x0005` both. Bits 3–15 are free for future hints.

Why it matters. LAST_CHUNK tells the ground where the file ends, so it is trimmed to its exact size.

Locate `bytes 8–23`
Where does this piece belong?

file_id `bytes 8–15 · i64`

Which file this piece belongs to.

How it's set. The row id from the sender's SQLite ledger, created the first time it sees a file path (folder scan or `ctl submit`).

How it can differ. Unique per file within one ledger. The same path keeps its id; a fresh ledger starts again at 1.

Why it matters. Many files can be in flight without mixing, and it is the number the cleaner quotes: "file 117, chunks 21–22".

chunk_id `bytes 16–19 · u32`

Which piece this is, counting from 0.

How it's set. From the sender's queue: a first pass counts 0 → N−1; a retransmit carries exactly the chunk the cleaner asked for. Manifest packets carry 0.

How it can differ. Different on every data packet of a file. Four bytes allow about 4.3 billion chunks.

Why it matters. The payload is written at `chunk_id × chunk size` and that chunk's bit is set, so arrival order never matters.

num_chunks `bytes 20–23 · u32`

How many pieces the whole file has.

How it's set. Computed by the sender: file size ÷ (chunk size − 32), rounded up, minimum 1. The same on every packet of the file.

How it can differ. Only if the chunk size changes (`ctl set-link --chunk-size`). Mid-file, the count no longer matches the ground's manifest, so those packets are rejected instead of scrambling the file.

Why it matters. The ground can size the bitmap from whichever packet arrives first, even if the filename packet was lost.

Arrival order doesn't matter

0	1	2	3	4	5	6	7
2nd	5th	lost	3rd	7th	1st	6th	4th

Big number: `chunk_id`. Small: when it arrived. Each payload is written at `chunk_id × chunk size` the moment it lands. Chunk 2 is just an unset bit until a retransmit fills it.

Verify `bytes 24–31`
Is it intact?

payload_len `bytes 24–27 · u32`

How many payload bytes follow the header.

How it's set. The encoder writes the real payload size: 16,160 for a full default chunk, less for the last chunk, the filename's length for a Manifest.

How it can differ. It varies by packet, but must always equal the datagram length minus 32.

Why it matters. A packet cut short or padded out is caught before anything else is trusted.

crc32 `bytes 28–31 · u32`

A checksum over the entire packet, header and payload, computed with this field set to zero.

How it's set. Computed last by the encoder, after every other byte is in place. The receiver zeroes the field, recomputes, and compares.

How it can differ. It changes with every packet's content. Flip one bit anywhere and the two values disagree.

Why it matters. It is the last gate before a byte touches the disk: a damaged packet is dropped and re-requested like a lost one.

A corrupted packet is treated exactly like a lost one.

parachuter never repairs damage. It only has to detect it; the retransmit loop does the recovery.

1. **Flight computer**
   Computes CRC-32 over header + payload with the CRC field zeroed, then writes it into bytes 28–31.
2. **The journey**
   Radio, relay satellite, modems, gateways. Cosmic-ray bit flips and radio bit errors happen here.
3. **Ground station**
   Recomputes the CRC over exactly what arrived and compares.

Match	Mismatch
accept, write, set the bit	drop it; the chunk stays a gap and the cleaner asks again

CRC-32 up close why parachuter carries its own checksum, and how much to trust it

Why not rely on UDP's own checksum?

- **Weak:** only 16 bits, and many error patterns cancel out.
- **Optional:** on IPv4 a sender can leave it off entirely.
- **Rebuilt mid-route:** gateways that repackage packets can compute a fresh one *after* the damage, approving corrupted data.

parachuter's CRC is computed once on the flight computer and checked once on the ground: end to end.

How good is CRC-32?

100%	100%	100%	1 in 4.3 B
of single-bit errors caught	of error bursts up to 32 bits caught	of odd numbers of flipped bits caught	heavier random damage that slips through

Hardware-accelerated on modern processors, and only 4 bytes. **It detects accidents, not attacks:** anyone can compute a CRC, so untrusted networks should wrap the link in an encrypted tunnel such as WireGuard.

§4 The manifest

One bit per chunk: the ground's to-do list.

117.incoming

The file's bytes. Each chunk is written at `chunk_id × size`; gaps stay zero until they are filled.

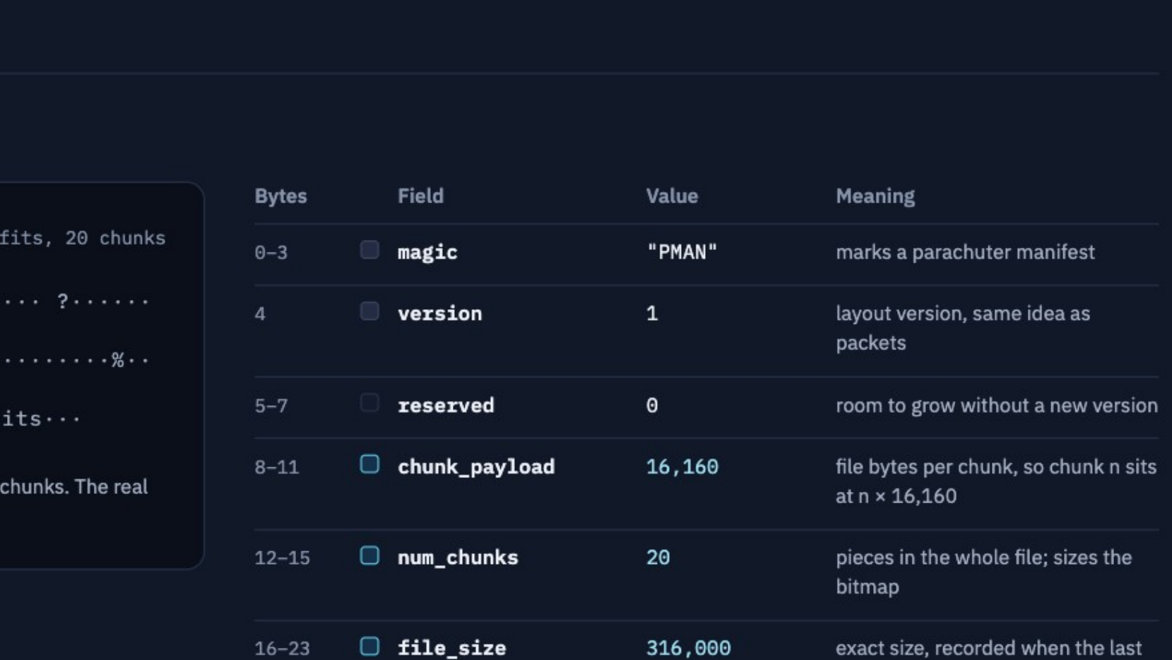
117.parmanifest

`PMAN · v1 · sizes · CRC`	`32 B`
`filename (UTF-8)`	`n B`
`bitmap 1101 1110 0111 …`	`⌈N/8⌉ B`

A 4 GB image in 16 KB chunks needs a bitmap of only **~32 KB**.

Crash-safe by construction. A chunk is written and synced to disk *before* its bit is set, and the manifest is replaced in one atomic rename. Lose power mid-flight, restart, and it resumes exactly where it stopped.

Watch the gaps close `real demo run · 3 MiB · 775 chunks · 20% loss`



Inside 117.parmanifest, byte by byte

```
$ xxd 117.parmanifest                          43 bytes · example: m31.fits, 20 chunks
0000   50 4D 41 4E 01 00 00 00 20 3F 00 00 14 00 00 00   PMAN···· ?······
0010   60 D2 04 00 00 00 00 00 08 00 00 00 E4 25 8B EE   `···········%··
0020   6D 33 31 2E 66 69 74 73 FB F9 0D                  m31.fits···
```

A fixed 32-byte header, then the filename, then the bitmap. The bitmap grows with the file: 1 byte per 8 chunks. The real filename is the full path the sender used; it is shortened here.

Bytes	Field	Value	Meaning
0–3	**magic**	"PMAN"	marks a parachuter manifest
4	**version**	1	layout version, same idea as packets
5–7	**reserved**	0	room to grow without a new version
8–11	**chunk_payload**	16,160	file bytes per chunk, so chunk n sits at n × 16,160
12–15	**num_chunks**	20	pieces in the whole file; sizes the bitmap
16–23	**file_size**	316,000	exact size, recorded when the last chunk lands
24–27	**name_len**	8	length of the filename that follows
28–31	**header_crc**	EE8B25E4	CRC-32 of the header: a damaged manifest is caught, not trusted
32–39	**filename**	m31.fits	from the filename packet; empty until it arrives
40–42	**bitmap**	FB F9 0D	one bit per chunk: 1 = on disk, 0 = missing

Reading the bitmap

	bit 0	bit 1	bit 2	bit 3	bit 4	bit 5	bit 6	bit 7
0xFB byte 40	1 chunk 0	1 chunk 1	0 chunk 2	1 chunk 3	1 chunk 4	1 chunk 5	1 chunk 6	1 chunk 7
0xF9 byte 41	1 chunk 8	0 chunk 9	0 chunk 10	1 chunk 11	1 chunk 12	1 chunk 13	1 chunk 14	1 chunk 15
0x0D byte 42	1 chunk 16	0 chunk 17	1 chunk 18	1 chunk 19	0 unused	0 unused	0 unused	0 unused

Chunk *n* lives in byte `n ÷ 8`, bit `n mod 8`, lowest bit first. The zeros read straight off as ranges, and that list is exactly what the cleaner sends: `2, 9–10, 17`.

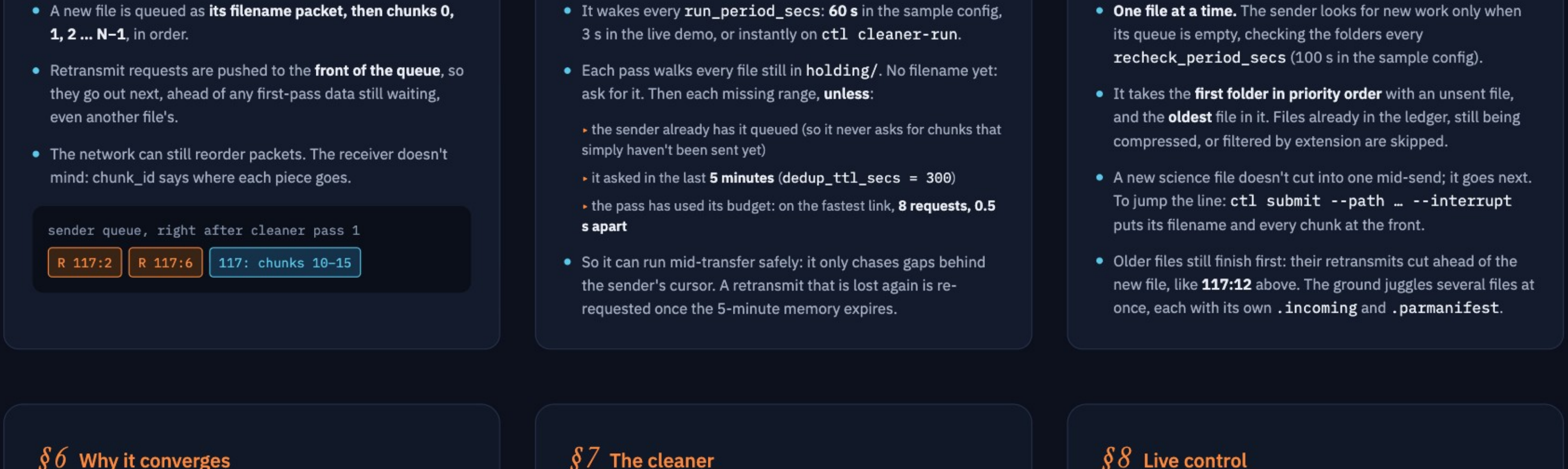
The life of a manifest

1. **Born with the first packet**
   Whichever arrives first for a new file_id, filename or data, creates it: all bits 0. If the filename came first, chunk size is unknown and saved as 0 until the first data chunk fills it in.
2. **Updated on every chunk**
   Write the bytes into `117.incoming`, sync to disk, set the bit, write the whole manifest to a temp file, then rename it over the old one. A crash leaves the old or the new manifest, never half of one.
3. **Sized by the last chunk**
   The LAST_CHUNK packet fixes the exact file size and the data file is trimmed to it.
4. **Deleted on completion**
   Every bit set and the name known: the data file is renamed to its real name in `downloads/` and the manifest is removed.

§5 Order of play

One packet at a time. Who goes next?

The sender keeps a single queue and sends one packet at a time, paced by the rate limiter. Three things decide what comes next: file order, the cleaner's retransmit requests, and your own commands.


file 117 chunk
file 118 chunk
filename packet
retransmit (R)
lost on the link
cleaner pass
asks 117: 2, 6
cleaner pass
asks 117: 12
cleaner pass
asks 118: 5
117 ✓
118 ✓
name 0 1 3 4 5 7 8 9 R 2 R 6 10 11 13 14 15 name 0 1 2 3 R 12 4 6 7 R 5 8 9
117 117 117 118
file 117 · filename, then chunks 0 → 15 in order
retransmits for 2 and 6 cut in ahead of chunk 10
file 118 · picked when the queue ran empty
117's last gap still jumps ahead of it
time: one packet at a time, paced by the rate limiter

What order do chunks go in?

- A new file is queued as **its filename packet, then chunks 0, 1, 2 … N−1**, in order.
- Retransmit requests are pushed to the **front of the queue**, so they go out next, ahead of any first-pass data still waiting, even another file's.
- The network can still reorder packets. The receiver doesn't mind: chunk_id says where each piece goes.

`sender queue, right after cleaner pass 1`
`R 117:2` `R 117:6` `117: chunks 10–15`

How often does the cleaner run?

- It wakes every `run_period_secs`: **60 s** in the sample config, 3 s in the live demo, or instantly on `ctl cleaner-run`.
- Each pass walks every file still in `holding/`. No filename yet: ask for it. Then each missing range, **unless**:
  - the sender already has it queued (so it never asks for chunks that simply haven't been sent yet)
  - it asked in the last **5 minutes** (`dedup_ttl_secs = 300`)
  - the pass has used its budget: on the fastest link, **8 requests, 0.5 s apart**
- So it can run mid-transfer safely: it only chases gaps behind the sender's cursor. A retransmit that is lost again is re-requested once the 5-minute memory expires.

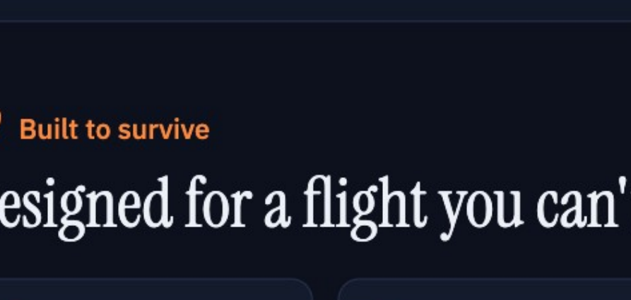
What if another file is waiting?

- **One file at a time.** The sender looks for new work only when its queue is empty, checking the folders every `recheck_period_secs` (100 s in the sample config).
- It takes the **first folder in priority order** with an unsent file, and the **oldest** file in it. Files already in the ledger, still being compressed, or filtered by extension are skipped.
- A new science file doesn't cut into one mid-send; it goes next. To jump the line: `ctl submit --path … --interrupt` puts its filename and every chunk at the front.
- Older files still finish first: their retransmits cut ahead of the new file, like **117:12** above. The ground juggles several files at once, each with its own `.incoming` and `.parmanifest`.

§6 Why it converges

Each pass shrinks the gap by the loss rate.

$$\mathrm{missing}(k) \approx N \times p^{k}$$



The price of loss is modest

Extra airtime ≈ `p ÷ (1 − p)`. Every chunk costs about one send, plus re-sends only for what was lost.

10% loss	20% loss	50% loss
+11%	+25%	+100%

Demo measured +25.4% at 20% loss.

§7 The cleaner

Three guards so it never floods the link.

1 **Check the live queue**
Ask the sender what it's already about to send, and skip those ranges.

2 **Remember recent asks**
Anything requested in the last 5 minutes is skipped. The memory is saved to disk, so it works even if the sender is unreachable.

3 **Respect each link's budget**
A cap on requests per pass, spaced out in time.

link	max per pass	spacing
hg-fastest	8	0.5 s
hg-fast	4	1 s
hg-slow	2	2 s

Example budgets, set per link in the config. Wakes every 60 s, or right now with `ctl cleaner-run`.

§8 Live control

Retune anything mid-flight. No restarts.

```
# link margin is dropping: slow down
$ ctl set-link --link hg-slow

# squeeze the throughput cap
$ ctl set-link --kbps 300

# smaller datagrams on a noisy link
$ ctl set-link --chunk-size 8192

# jump the queue with one file
$ ctl submit --path m31.fits --interrupt

# hold everything
$ ctl set-state paused
```

- Every daemon listens on a **Unix socket** speaking length-prefixed JSON, so any language can drive it.
- Edit the TOML config and it **hot-reloads in about a second**.
- `parachuter monitor` shows a live table of every file in flight.

§9 Built to survive

Designed for a flight you can't go and fix.

Checked twice	Power-cut proof	Any order	Multicast-ready	Every file tracked	Science first
CRC-32 on every packet and on every manifest header.	Sync, then atomic rename. Restart resumes mid-file.	Chunks land at their offset. The name can arrive last.	The receiver joins multicast groups by itself.	A SQLite ledger, with CSV snapshots for the ground.	Ordered priority folders, plus a separate debug queue.

Part 2 · new in v0.2.0

Files lost whole

When an outage swallows every packet of a file, the ground never learns it existed. The payload's own ledger, downlinked like any other file, brings it back.

§10 The gap

Two kinds of outage. Only one was covered.

Outage cuts through a file

The cleaner repairs it.

`117.parmanifest exists` `outage = one gap: chunks 8–16`

At least one packet landed, so the ground has a manifest and knows the file's size. Once the link is back, the cleaner asks for the missing range in a single request.

Outage swallows every packet

Nobody noticed. Until v0.2.

Payload ledger
`118 · m31_002.fits.bz2`
`queued ✓ · "sent"`

`GROUND holding/`
`no 118.parmanifest`
`nothing to chase`

No packet landed, so there is no manifest and nothing for the cleaner to find. The sender already counts the file as sent and never picks it up again. Files still queued in memory when the sender restarts end up the same way.

§11 The fix

The payload's ledger is the acknowledgement.

Nothing new flies up to the payload. The sender already knows everything it sent; the ground only has to compare that list with what landed.

1 — Payload · every 6 h
Snapshot the ledger
The sender writes `files_sent_list_<time>.csv`: every file's full path, size, whether it still exists, and `queued_at`.

2 — The link
Downlink it like any file
It goes into the **first** priority folder (e.g. `sql_status/`), so it jumps ahead of science. If an outage eats it, the cleaner or the next snapshot covers it.

3 — Ground · every 6 h
Compare with final_dir
The cleaner reads the newest snapshot and checks every row at its mirrored path. Files land there only when complete, so the folder *is* the record.

4 — Ground → sender
Resend what's lost
A file lost whole is requested again in full, at the **back** of the queue, at most once per cycle, until it lands.

On the payload	`/data/qsc/science/m31_002.fits.bz2`
On the ground	`/downloaded_files/data/qsc/science/m31_002.fits.bz2`

§12 One pass, row by row

Every file in the snapshot gets a verdict.

`…/status_dumps/files_sent_list_1790376000.csv` verdict →

id	file_name	queued	verdict
117	/data/qsc/science/m31_001.fits.bz2	09:12	complete
118	/data/qsc/science/m31_002.fits.bz2	09:40	lost whole → resend
119	/data/qsc/dark/dark_0007.fits.bz2	10:05	in flight
120	/data/qsc/flat/flat_0012.fits.bz2	13:58	still queued
121	/data/qsc/science/m31_003.fits.bz2	14:51	too recent

Pass run at 15:30 with a 3 h grace period. Only file 118 is requested again.

What each verdict means

`complete`	At its mirrored path with the ledger's size.
`in flight`	Partial, in `holding/`: the cleaner owns it. Flagged if unchanged for longer than the grace period.
`still queued`	Waiting in the sender's live queue.
`too recent`	Queued under 3 h ago: give it time.
`gone on payload`	Never arrived and deleted on the payload. Reported; cannot be resent.
`size mismatch`	Landed at the wrong size. Reported, not resent: the payload copy likely changed.
`lost whole`	Requested again in full, at the back of the queue.

§13 How long it takes

Worst case: about 12 hours.

`until the next ledger snapshot · ≤ 6 h` `until the next reconcile pass · ≤ 6 h`

`0 h · file lost` `6 h · snapshot written and downlinked` `12 h · found, requested again`

The 3 h grace only holds back files queued just before a pass, so it never extends the worst case. Typical detection is much sooner. Shorten `status_dump_period_secs` and `reconcile_period_secs` to find out faster; the snapshot costs about 150 bytes per file.

Settings and commands

`final_dir`	where files land, e.g. /downloaded_files
`status_dump_period_secs`	21600 · payload snapshot
`reconcile_period_secs`	21600 · ground pass
`reconcile_grace_secs`	10800 · grace

```
$ ctl -s cleaner.sock reconcile-run
$ ctl -s cleaner.sock status
"last_reconcile": { "complete": 3, "missing": 1, "requested": 1 … }
```

Keep `final_dir` as an archive: copy files out, never move them, or they will be requested again.

`$ cargo install parachuter` `$ cargo add parachuter`

`github.com/emaadparacha/parachuter`

Emaad Paracha · 2026
Free for non-commercial use